

Coding Nexus · [Follow publication](#)

★ Member-only story

A Simple Trick That Cuts Your LLM Token Costs by 50%: Meet TOON

4 min read · 6 days ago



Algo Insights

Following ▾



Listen



Share



More

If you've ever played around with large language models (LLMs), you know the pain of **token costs**.

Every minor character you send to an API — including commas, braces, and quotes — costs you Money. And the biggest culprit? Good old **JSON**.

Don't get me wrong, **JSON** is great for code. But when you're sending it to an AI, it's like wrapping your data in bubble wrap made of dollar bills. You pay for every quote, every key, every bracket.

A few weeks ago, I came across this format called **TOON** — short for **Token-Oriented Object Notation** — and honestly, it feels like cheating. It's essentially JSON, minus all the extra clutter.

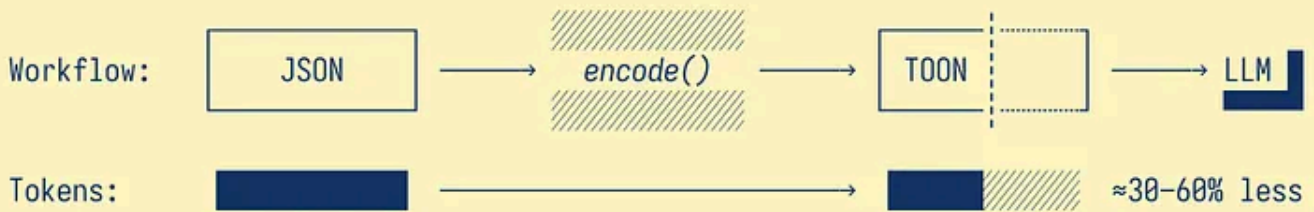
And the best part? It can **reduce your token usage by up to 50%**.

Let me explain what I mean.

Token-Oriented Object Notation



A Compact, human-readable, schema-aware serialization for LLM prompts



	avg tokens	retrieval accuracy	best for
TOON		73.9%	repeated structure • tables
JSON		69.7%	varying fields • deep trees

. . .

JSON vs TOON

Here is a simple example. This is your standard JSON:

```
{
  "users": [
    { "id": 1, "name": "Alice", "role": "admin" },
    { "id": 2, "name": "Bob", "role": "user" }
  ]
}
```

And here's the same data in TOON:

```
users[2]{id,name,role}:
  1,Alice,admin
  2,Bob,user
```

That's it. No curly braces, no quotes — just a clean, readable structure.

You might expect the model to get confused, but no — it actually *understands it better*.

When you tokenise both versions, the TOON one uses **nearly half the tokens**. That's Money saved *each time you hit "send"*.

. . .

What Makes TOON So Efficient?

The trick is that TOON isn't trying to be fancy. It's simply *minimal*.

It removes all the extra symbols JSON requires and uses indentation and headers instead. Think of it as a blend of YAML and CSV — compact and straightforward.

Here's the vibe:

- Use **indentation instead of braces**.
- Use **headers for columns** (like CSV).
- Keep arrays **uniform** — meaning each item has the same fields.

That's why it's ideal for tasks such as user lists, logs, or data tables.

If your data resembles a spreadsheet, TOON will easily master JSON.

. . .

Let's Try It in Code

You can install it like this:

```
npm install @toon-format/toon
```

Then:

```
import { encode } from '@toon-format/toon'

const data = {
  users: [
    { id: 1, name: 'Alice', role: 'admin' },
```

```
{ id: 2, name: 'Bob', role: 'user' }  
]  
}  
console.log(encode(data))
```

And the output?

```
users[2]{id,name,role}:  
  1,Alice,admin  
  2,Bob,user
```

Clean. Easy. Reversible.

You can even convert it back to JSON if needed. No magic, no data loss.

• • •

The Numbers Don't Lie

Someone actually benchmarked it, and the results were pretty wild:

Format	Accuracy	Tokens Used	Token Savings
TOON	73.9%	2,744	-39.6%
JSON (compact)	70.7%	3,081	-22.9%
YAML	69.0%	3,719	-18.6%
XML	67.1%	5,167	-47.0%

So yeah — fewer tokens lead to *better model performance*. Who knew ditching braces could make an AI smarter?

• • •

When to Use TOON (and When Not To)

Here's the honest bit — TOON isn't perfect for everything.

It's ideal for structured, repetitive data, such as rows, logs, or tables. It's less effective for complex, deeply nested structures, such as config files or unpredictable formats.

Basically, if your data looks like a spreadsheet — TOON.

If it looks like an abstract tree — probably stick to JSON.

• • •

Bonus: You Can Use Tabs or Pipes Too

By default, TOON separates fields with commas. But if you want to get fancy, you can switch to tabs or pipes — they tokenize better and look cleaner.

```
encode(data, { delimiter: '\t' })
```

Result:

```
users[2]{id      name      role}:  
  1      Alice   admin  
  2      Bob     user
```

Simple tweak, slightly reducing token count.

• • •

CLI Goodies

You can also convert data directly from the command line.

```
npx @toon-format/cli input.json -o output.toon
```

Or reverse it:

```
npx @toon-format/cli data.toon -o output.json
```

And if you're curious how much you're saving:

```
npx @toon-format/cli data.json --stats
```

It'll show you precisely how many tokens you're saving. (It's oddly satisfying to see those numbers decrease.)

. . .

Why I'm Excited About This

LLMs are becoming larger and more powerful, but they also *require more resources*. We continue feeding them additional data — user histories, context, metadata, embeddings — and the token costs keep climbing.

Formats like TOON seem like a small but clever protest against that. It's not an overcomplicated compression method — it's simply **text done properly**.

You achieve human readability *and* lower costs. And that's rare.

. . .

The TL;DR

- JSON is too wordy for LLMs
- TOON is a cleaner, leaner format made for token efficiency
- Saves up to 50% tokens
- Keeps data readable and reversible
- Great for tables, lists, and logs

Example:

```
{ "id": 1, "name": "Alice", "role": "admin" }
```

Becomes:

```
users[1]{id,name,role}:  
1,Alice,admin
```

Same meaning. Half the price.

• • •

Final Thoughts

If you're working with LLMs, this could be one of the simplest wins you can achieve.

Simply changing your data format could allow you to send twice as much context — or cost half as much for the same task.

No new model, no new API—just better notation.

Give it a try:

```
npm install @toon-format/toon
```

Then run:

```
npx @toon-format/cli data.json --stats
```

If you're like me, you'll probably smile when you see how many tokens you've just saved.

[Llm](#)[Llm Applications](#)[AI](#)[Ai Agent](#)[Ai Tools](#)[Follow](#)

Published in Coding Nexus

8.6K followers · Last published 6 hours ago

Coding Nexus is a community of developers, tech enthusiasts, and aspiring coders. Whether you're exploring the depths of Python, diving into data science, mastering web development, or staying updated on the latest trends in AI, Coding Nexus has something for you.

[Following](#)

Written by Algo Insights

3.6K followers · 6 following

Algo Insights: Unlocking the Future of Trading with Code, Data, and Intelligence.
https://linktr.ee/Algo_Insights

Responses (3)



Bgerby

What are your thoughts?



Ali Farhat
3 days ago



Great article. Would you be open to adding our JSON to TOON converter as a reference?

<https://scalevise.com/json-toon-converter>

It runs fully client side, stores nothing outside the browser and includes a small history feature that helps developers... [more](#)



5

[Reply](#)



Susan Apfel
4 days ago



It looks like information compression. If the llm could understand a compressed form of prompt also output in compressed form, we could save token. What do you think if it's reasonable or possible? Or we could just train llm this way only in compressed form?



2



1 reply

[Reply](#)



lillian cipciriuc
13 hours ago



Is there a way to use hybrid declarations? Like toon and json in one pass, still it's a saving. For the tabular data -toon, for the branching structure -json, but both in one?



[Reply](#)

More from Algo Insights and Coding Nexus

 In Coding Nexus by Algo Insights

DeepAgent: How the New AI Agent Learns, Thinks, and Builds Its Own Tools

When I first read about DeepAgent, I couldn't help but pause and wonder — what does it truly mean for an AI to discover its own tools?

★ Nov 1 🖱️ 180 💬 2

🔖⁺ ⋮

 In Coding Nexus by Code Coup

Claude Desktop Might Be the Most Useful Free Tool You'll Install This Year

I didn't expect much when I first saw the announcement for Claude Desktop. Another AI wrapper, I thought. Maybe with a shiny UI.

★ Oct 23 🖱️ 1K 💬 41



 In Coding Nexus by Civil Learning

The Guy Who Let ChatGPT Trade for Him—and Somehow It Worked

You know how everyone says, "Don't let AI touch your Money"? Well, someone on Reddit decided to ignore that.

★ Oct 8 🖱️ 278 💬 9





In Coding Nexus by Algo Insights

4 Open-Source Tools That Made Me Rethink My Dev Setup

I've been coding for a while now. Most of the tools we use every day... they've been the same for years. Editors, browsers, frameworks. Just...



Sep 3



459



9



See all from Algo Insights

See all from Coding Nexus

Recommended from Medium

 Dr Adrian Lucas Malec

How I Built Lightning-Fast Vector Search for Legal Documents

From 53 to 2,880 queries/sec with USearch

Oct 19  395  5



 In AIGuys by Vishal Rajput 

Kimi K2 Just Killed OpenAI, & Claude

If one thing tech bros are better than at building AI, it is creating hype around AI. And the release of Kimi K2 just proved the opposite...



Nov 8



515



17



In AI Software Engineer by Joe Njenga

I Tested New Cursor 2.0 Features (And Discovered AI Coding Not Seen Before)

Cursor 2.0 is about to fire more developers than we thought possible. If you're in doubt, see it in action here, then give it a try.



5d ago



64



In Medialesson by Marius Schröder

JSON vs TOON—A new era of structured input?

Why structure matters more than ever

Nov 3  797  42



 In AI Advances by Dr. Leon Eversberg 

PDF to Markdown Conversion Benchmark—Part 3

Benchmarking 8 Python tools for LLM, RAG & agentic workflows

★ Nov 9  358  5





In Coding Nexus by Code Coup

Google DS-STAR: The AI Agent That Automates Data Science from Analysis to Visualization

If you've ever juggled five different file types while trying to answer a simple data question, you'll appreciate what Google has just...



6d ago



50



1



See more recommendations